



Home Work

Generative AI

multi agent

1. فایل تمرین را در پنل خود آپلود کنید.
2. title فایل پروژه به صورت (نام پروژه + نام و نام خانوادگی) به انگلیسی باشد.
3. در صورتی که سوال و یا ابهامی دارید در گروه چت تلگرامی بپرسید.

طراحی سیستم پشتیبانی هوشمند با LangGraph

۱. مرور کلی و هدف پروژه

در این پروژه نهایی، شما فراتر از یک چتبات ساده می‌روید و یک سیستم پشتیبانی مشتریان چند-عاملی طراحی می‌کنید. برخلاف مدل‌های ساده که سعی می‌کنند به همه سوالات با یک پرامپت پاسخ دهند، این سیستم مانند یک شرکت واقعی عمل می‌کند: دارای یک "مسئول پذیرش" است که درخواست‌ها را بررسی کرده و آن‌ها را به "متخصصان" مربوطه (مثلاً بخش مالی یا بخش فنی) ارجاع می‌دهد. همچنین، این سیستم باید هوشمند باشد و در صورتی که کاربر عصبانی است، به جای پاسخ خودکار، کار را متوقف کرده و به یک اپراتور انسانی ارجاع دهد.

۲. معماری سیستم

شما باید با استفاده از کتابخانه‌ی LangGraph، سیستمی با گره‌ها و جریان کاری زیر پیاده‌سازی کنید:

گره ۱: ایجنت دسته‌بندی و توزیع (The Triage Agent)

- نقش: مسئول پذیرش.
- وظیفه: پیام ورودی کاربر را تحلیل کرده و آن را به یکی از سه دسته زیر طبقه‌بندی می‌کند:
 - BILLING: مسائل مربوط به پرداخت، اشتراک و بازگشت وجه.
 - TECHNICAL: مشکلات فنی، باگ‌ها و سوالات محصول.
 - GENERAL: سوالات عمومی و احوال‌پرسی.
- خروجی: یک تصمیم ساختاریافته که مشخص می‌کند گره بعدی کدام است.

گره ۲: متخصص امور مالی (Billing Specialist)

- نقش: رسیدگی به درخواست‌های حساس مالی.
- ابزارها:
 - `check_subscription_status(user_id)`: بررسی فعال یا منقضی بودن اشتراک کاربر.
 - `process_refund(transaction_id)`: به صورت Mock، تایید بازگشت وجه.
- رفتار: این ایجنت نباید به سوالات فنی پاسخ دهد. اگر سوال نامرتبیطی دریافت کرد، باید درخواست را دوباره به "تریاز" برگرداند.

گره ۳: متخصص پشتیبانی فنی (Technical Support)

- نقش: حل مشکلات نرم‌افزاری.

- ابزارها:

- `search_knowledge_base(query)`: استفاده از سیستم RAG برای جستجو در یک فایل

راهنما یا داکيومنت فنی.

- رفتار: اگر پاسخ را در داکيومنت‌ها پیدا نکرد، نباید توهم داشته باشد و باید اعلام کند که پاسخ را نمی‌داند.

گره ۴: محافظ تحلیل احساسات (Sentiment Guardrail)

- نقش: کنترل کیفیت و نظارت.

- وظیفه: قبل از ارسال پاسخ نهایی ایجنت‌ها به کاربر، این گره لحن پیام کاربر را بررسی می‌کند.

- منطق:

- اگر احساسات منفی/خشمگین باشد (مثلاً: "سرویس شما افتضاح است!"): اجرای گراف را

متوقف کرده و منتظر دخالت انسان می‌ماند.

- اگر احساسات خنثی/مثبت باشد: پاسخ تولید شده را به کاربر نمایش می‌دهد.

۳. مراحل پیاده‌سازی فنی

مرحله ۱: تعریف وضعیت (State)

ابتدا باید وضعیت گراف را با استفاده از TypedDict تعریف کنید تا بتوانید اطلاعات را بین ایجنت‌ها جابجا کنید:

```
from typing import TypedDict, List, Annotated
import operator

class SupportState(TypedDict):
    messages: Annotated[List[str], operator.add] # Chat History
    user_id: str
    sentiment: str # "Positive", "Neutral", "Negative"
    department: str # "Billing", "Technical", "Triage"
    next_step: str # Control flow variable
```

مرحله ۲: ساخت ایجنت دسته‌بندی

با استفاده از قابلیت with_structured_output در لنگ‌چین، ایجنت تریاژ را طوری طراحی کنید که خروجی آن حتماً یک JSON باشد که قصد کاربر را مشخص می‌کند.

مرحله ۳: پیاده‌سازی نظارت انسانی (Human-in-the-Loop)

این بخش اصلی پروژه است. شما باید از قابلیت checkpoint در LangGraph استفاده کنید.

- زمانی که "محافظ احساسات" خشم کاربر را تشخیص داد، باید با دستور graph.interrupt() اجرا را متوقف کنید.

- یک تابع ساده بنویسید که نقش "مدیر پشتیبانی" را بازی کند. این تابع باید بتواند وضعیت را بررسی کرده و پاسخ ایجنت را تایید یا رد کند (و یا یک پاسخ دستی جایگزین کند).

۴. سناریوهای مورد انتظار برای تحویل

در فایل نوت‌بوک تحویلی، شما باید عملکرد سیستم خود را در ۳ سناریوی زیر نمایش دهید:

1. مسیر موفق فنی:

- کاربر: "چطور می‌توانم رمز عبورم را ریست کنم؟"
- جریان سیستم: تریاژ < متخصص فنی < جستجوی RAG
- پاسخ نهایی: "برای تغییر رمز عبور روی لینک زیر کلیک کنید..."

2. عملیات حساس مالی:

- کاربر: "اشتراک من کار نمی‌کند. آی‌دی من ۱۲۳۴۵ است."
- جریان سیستم: تریاژ < متخصص مالی < ابزار check_subscription
- پاسخ نهایی: "اشتراک شما منقضی شده است."

3. ارجاع به انسان

- کاربر: "پول من را دزدیدید! این سرویس به درد نمی‌خورد! می‌خواهم با مدیر حرف بزنم."
- جریان سیستم: تریاژ تشخیص می‌دهد موضوع "مالی" است < اما "محافظ احساسات" خشم را تشخیص می‌دهد < توقف سیستم.
- اقدام شما: شما باید با استفاده از متد update_state یک پاسخ دستی از طرف مدیر وارد کنید: "من مدیر ارشد هستم. عذرخواهی می‌کنم، مشکل شما را شخصاً پیگیری خواهم کرد."

نحوه‌ی تحویل:

کد کامل پیاده‌سازی شده در قالب فایل `ipynb` به همراه تصویر گراف تولید شده (با استفاده از `draw_mermaid_png()` ارسال شود).